

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE NAME:** Artificial Intelligence

**COURSE CODE:** CSA17

---

### COURSE OUTCOME COVERED

**CO1:** Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

*Assessment Tool Weightage: 50%*

---

**QUESTIONS: 5**

**Duration : 1 Hour**  
**Total Marks:50**

Annexure A

### Analytical Problem Solving

---

#### ***Code of Conduct:***

*I ROSHITHA. Y (192424400) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

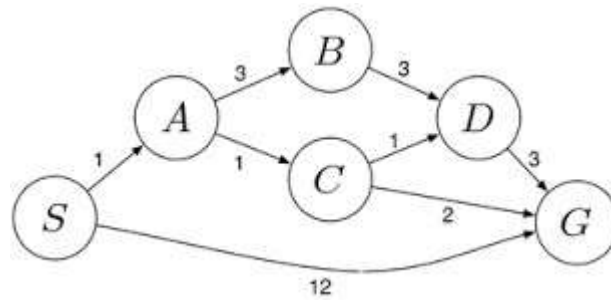
***Signature of the student:***

## Annexure A

### **Analytical Problem Solving**

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
  - i. What are the percept's for this agent?
  - ii. Characterize the operating environment.
  - iii. What are the actions the agent can take?
  - iv. How can one evaluate the performance of the agent?
  - v. What sort of agent architecture do you think is most suitable for this agent? Why?
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.
4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.
5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



**RUBRICS FOR CALCULATION:**

| Criteria                  | Weightage | Level 4<br>(Exemplary)                                                            | Level 3<br>(Proficient)                   | Level 2<br>(Developing)                    | Level 1<br>(Beginning)                        |
|---------------------------|-----------|-----------------------------------------------------------------------------------|-------------------------------------------|--------------------------------------------|-----------------------------------------------|
| Problem Understanding     | 20%       | Clearly interprets problem, identifies all parameters and requirements accurately | Understands problem with minor gaps       | Partial understanding; misses key elements | Misinterprets or unable to understand problem |
| Method Selection          | 20%       | Selects most appropriate method/formula with strong justification                 | Selects correct method with minor errors  | Inappropriate or partially correct method  | Unable to select method                       |
| Solution Process          | 25%       | Logical, systematic steps with correct approach throughout                        | Mostly correct steps with minor mistakes  | Disorganized steps; multiple errors        | No clear process                              |
| Accuracy of Calculation   | 20%       | All calculations accurate; correct final answer                                   | Minor calculation errors                  | Frequent errors; incorrect final answer    | No valid calculations                         |
| Interpretation of Results | 15%       | Clearly interprets results with units and engineering relevance                   | Basic interpretation with minor omissions | Limited or unclear interpretation          | No interpretation                             |
| Problem Understanding     | 20%       | Clearly interprets problem, identifies all parameters and requirements accurately | Understands problem with minor gaps       | Partial understanding; misses key elements | Misinterprets or unable to understand problem |

# 1. Water Jug Problem

The two jugs hold 4 gallons and 3 gallons respectively, with no calibration marks, so the only reliable operations are: fill a jug completely from the pump, empty a jug fully, and pour from one jug into another until either the source is empty or the destination is full.

A shortest route to 2 gallons in the 4-gallon jug is found by first filling the 4-gallon jug completely, then pouring it into the 3-gallon jug until the 3-gallon jug is full. This leaves 1 gallon in the 4-gallon jug and 3 gallons in the small jug. Emptying the 3-gallon jug and pouring that leftover 1 gallon into it gives 1 gallon in the small jug. Filling the 4-gallon jug once more and topping up the 3-gallon jug (which needs only 2 more gallons to become full) removes exactly 2 gallons from the 4-gallon jug, leaving precisely 2 gallons behind.

Step-by-step state trace (Jug-A = 4 gal, Jug-B = 3 gal):

- (0,0) -> Fill A -> (4,0)
- (4,0) -> Pour A into B -> (1,3)
- (1,3) -> Empty B -> (1,0)
- (1,0) -> Pour A into B -> (0,1)
- (0,1) -> Fill A -> (4,1)
- (4,1) -> Pour A into B -> (2,3)

Final state: Jug-A contains exactly 2 gallons, satisfying the goal. This corresponds to the BFS search tree explored by the accompanying Python program, which searches the state space of (Jug-A, Jug-B) pairs breadth-first so that the first path reaching a state with Jug-A = 2 is guaranteed to be shortest.

# 2. Mars Rover Agent

## i. Percepts

The rover perceives its surroundings through a camera feed, a spectrometer reading of mineral composition, a soil probe reading, ambient temperature, terrain classification, obstacle-detection flags, and dust-storm alerts relayed from onboard weather sensors.

## ii. Operating Environment

The environment is partially observable (sensors only cover the rover's immediate vicinity), sequential in the sense that current actions affect future terrain choices, dynamic due to weather and lighting changes, mostly continuous in its state variables, and effectively unknown in fine detail since the surface has not been mapped at the resolution the rover encounters it. It also has significant communication latency with mission control on Earth.

## iii. Actions

Available actions include driving forward or backward, rotating in place, photographing the surface, extracting a core sample, running a spectral analysis on collected material, and uplinking processed data to an orbiting relay satellite.

#### iv. Performance Evaluation

Performance is measured by the number of samples successfully analysed, remaining battery reserve after each task, avoidance of terrain hazards, accuracy of the spectral data gathered, and whether the rover completes its assigned traverse within the energy budget allotted for that Martian day (sol).

#### v. Suitable Agent Architecture

A model-based, goal-driven architecture is appropriate here rather than a purely reactive one, because the rover must track an internal model of terrain already surveyed and samples already collected in order to decide what remains to be done. Pure reflex behaviour is retained only for immediate hazard avoidance (e.g. halting on obstacle detection), while the higher-level goal component sequences driving, sampling and communication actions to satisfy the mission's overall objective within its energy constraints.

### 3. 8-Queens Problem

State representation: a partial or complete assignment of queens to columns, where each state records, for every column already filled, the row the queen occupies in that column.

Initial state: an empty board with zero queens placed.

Successor function: given a state with queens placed in the first  $k$  columns, generate one successor for every row in column  $k+1$  that is not attacked by any existing queen (same row or either diagonal).

Goal test: exactly 8 queens are placed, one per column, with no two sharing a row or a diagonal.

Path cost: uniform (each placement counts as one step); since only a valid final configuration matters, path cost plays no role in preferring one solution over another.

Search strategy used: backtracking depth-first search accelerated with bitmasks, where the columns already used, and the two diagonal directions, are each tracked as a single integer bitmask so that safe positions for the next queen can be computed with bitwise operations instead of a per-queen conflict loop. This is functionally equivalent to classic backtracking but prunes the search space faster.

### 4. OLA Cab Booking Agent

This is a goal-based agent whose goal is to secure a booking that best matches the customer's stated cab-type preference while also weighing driver rating and estimated arrival time, rather than accepting the very first available cab.

Pseudocode:

- CabBookingAgent(source, destination, preferred\_type):
- cabs <- FetchNearbyCabs(source)
- if cabs is empty: return "No cabs available"
- candidates <- Filter(cabs, type = preferred\_type)
- if candidates is empty: candidates <- cabs
- best <- Cab in candidates maximizing (rating\*10 - eta)
- fare <- best.rate\_per\_km \* EstimateDistance(source, destination)
- return Confirm(best, fare)

Scoring both ETA and driver rating (rather than ETA alone) lets the agent trade off a slightly longer wait for a materially better-rated driver, which is closer to how a rational customer-facing agent should behave.

## 5. Uniform Cost Search - Logistics Network

UCS expands the frontier node with the lowest cumulative cost recorded so far, which guarantees the first time the goal is popped from the frontier, the cost is optimal (assuming non-negative edge weights, which all these routes satisfy)

Trace of the search:

- Start: S, cost 0. Frontier after expansion: A (cost 1), G (cost 12).
- Expand A (cost 1): frontier gains C (cost 2) and B (cost 4); G (12) still pending.
- Expand C (cost 2): frontier gains D (cost 3) and an alternate G (cost 4).
- Expand D (cost 3): frontier gains another G route (cost 6), which is worse than the existing G (4) and is discarded.
- Expand G (cost 4): goal reached, search terminates.

Least-cost path: S → A → C → G, with total cost  $1 + 1 + 2 = 4$ . The direct S → G edge (cost 12) and the path through B are both more expensive and are correctly ruled out by UCS.

